

ESPECIFICACIÓN TÉCNICA DE LA API REST - RESTAURANTE SILVA

1. Visión General del Sistema

La API REST del Sistema Silva provee la capa de backend para la gestión del menú, catálogo de platillos y auditoría de cambios en la base de datos SQL Server. Está construida sobre .NET Core / C# y utiliza Entity Framework Core para el mapeo objeto-relacional (ORM).

Información del Servidor

- **URL Base (Desarrollo Local):** http://localhost:5195/api
- **Formato de Intercambio de Datos:** JSON (application/json)
- **Manejo de Errores Standard:**
 - 200 OK: Operación exitosa.
 - 201 Created: Recurso creado exitosamente.
 - 400 Bad Request: Datos de entrada inválidos o faltantes.
 - 404 Not Found: El recurso solicitado no existe.
 - 500 Internal Server Error: Error no controlado en el servidor o la BD.

2. Endpoints del Módulo de Platillos (/api/platillos)

2.1. Obtener Menú Público

- **Método HTTP:** GET
- **Ruta:** /api/platillos/menu-publico
- **Acceso:** Público
- **Descripción:** Ejecuta una consulta sobre la vista vw_MenuPublico de la base de datos. Retorna únicamente aquellos platillos que tienen el estado marcado como 'Disponible'.
- **Ejemplo de Respuesta (200 OK):**

JSON

```
[
  {
    "idPlatillo": 1,
    "nombre": "Nachos Supreme",
    "descripcion": "Crujientes tortillas de maíz con queso fundido, carne de res, guacamole y crema.",
    "precio": 8.50,
```

```

    "categoria": "Entradas",
    "imagenUrl": "https://ejemplo.com/imagenes/nachos.jpg",
    "tiempoPreparacion": 12,
    "estado": "Disponible"
  },
  {
    "idPlatillo": 3,
    "nombre": "Hamburguesa Silva Double",
    "descripcion": "Doble carne de res, queso mozzarella, lechuga y pan brioche. Incluye papas.",
    "precio": 12.00,
    "categoria": "Platos Fuertes",
    "imagenUrl": "https://ejemplo.com/imagenes/hamburguesa.jpg",
    "tiempoPreparacion": 20,
    "estado": "Disponible"
  }
]

```

2.2. Obtener Todos los Platillos (Gestión Interna)

- **Método HTTP:** GET
- **Ruta:** /api/platillos
- **Acceso:** Privado / Requerido para Administración
- **Descripción:** Retorna el listado completo de platillos en la base de datos independientemente de su estado (Disponible, Agotado, Desactivado).
- **Ejemplo de Respuesta (200 OK):** Retorna la lista JSON de platillos.

2.3. Obtener Platillo por ID

- **Método HTTP:** GET
- **Ruta:** /api/platillos/{id}
- **Parámetros de Ruta:** id (int, obligatorio) - ID del platillo a consultar.
- **Descripción:** Busca y retorna la información detallada de un solo platillo.
- **Ejemplo de Respuesta (200 OK):**

JSON

```

{
  "idPlatillo": 1,
  "idCategoria": 1,
  "nombre": "Nachos Supreme",
  "descripcion": "Crujientes tortillas de maíz con queso fundido, carne de res...",
  "precio": 8.50,
  "estado": "Disponible"
}

```

```
}
```

2.4. Crear un Nuevo Platillo

- **Método HTTP:** POST
- **Ruta:** /api/platillos
- **Cuerpo de la Petición (Request Body):**

JSON

```
{  
  "idCategoria": 2,  
  "idUsuarioUltimaModif": 1,  
  "nombre": "Pizza Pepperoni Artesanal",  
  "descripcion": "Masa madre, salsa de tomate casera, queso mozzarella y pepperoni.",  
  "precio": 13.50,  
  "imagenUrl": "https://ejemplo.com/imagenes/pizza.jpg",  
  "tiempoPreparacion": 18,  
  "estado": "Disponible"  
}
```

- **Descripción:** Registra un nuevo platillo en el catálogo. Automáticamente asigna la fecha actual y activa el trigger de auditoría si corresponde.
- **Ejemplo de Respuesta (201 Created):** Retorna el objeto creado con su idPlatillo autogenerated.

2.5. Actualizar un Platillo

- **Método HTTP:** PUT
- **Ruta:** /api/platillos/{id}
- **Parámetros de Ruta:** id (int, obligatorio)
- **Cuerpo de la Petición (Request Body):**

JSON

```
{  
  "idPlatillo": 3,  
  "idCategoria": 2,  
  "idUsuarioUltimaModif": 2,  
  "nombre": "Hamburguesa Silva Double Special",  
  "descripcion": "Doble carne de res, queso mozzarella, tocino y pan brioche.",  
  "precio": 12.50,  
  "imagenUrl": "https://ejemplo.com/imagenes/hamburguesa.jpg",  
  "tiempoPreparacion": 20,  
}
```

```
"estado": "Disponible"  
}
```

- **Descripción:** Actualiza las propiedades de un platillo existente. Al realizar el cambio, el trigger de la BD captura los valores anteriores y los nuevos en la tabla de auditoría.

2.6. Eliminar o Desactivar Platillo

- **Método HTTP:** DELETE
- **Ruta:** /api/platillos/{id}
- **Parámetros de Ruta:** id (int, obligatorio)
- **Descripción:** Realiza la eliminación del registro (o cambia su estado a Desactivado en caso de implementar borrado lógico).

3. Endpoints del Módulo de Auditoría (/api/auditoria)

3.1. Obtener Reporte de Auditoría

- **Método HTTP:** GET
- **Ruta:** /api/auditoria
- **Acceso:** Solo Administradores
- **Descripción:** Consulta la vista vw_ReporteAuditoria para extraer el historial detallado de modificaciones realizadas sobre las tablas del sistema.
- **Ejemplo de Respuesta (200 OK):**

JSON

```
[  
  {  
    "idAuditoria": 1,  
    "tablaAfectada": "Platillos",  
    "accion": "UPDATE",  
    "idRegistroAfectado": 3,  
    "campoModificado": "precio",  
    "valorAnterior": "11.50",  
    "valorNuevo": "12.00",  
    "fechaCambio": "2026-09-01T14:30:00",  
    "usuarioResponsable": "Carlos Mendoza"  
  }  
]
```